

Assignment 2 Priority Email GenAI Analysis

1. GenAI Used

The two Generative AIs that I used for this assignment are Perplexity AI accessed through its website (perplexity.ai) through the base model and Google Search AI accessed through google.com using AI search mode.

2. Code Analysis

Correctness: Both programs implement a list-based MaxHeap and correctly prioritize emails by sender category and newest date. Google Search uses standard input, while Perplexity opens a file using `argv[1]`. Both handle insertion, removal, and viewing the highest-priority email.

Execution Time: Both programs have $O(\log n)$ insertion and removal, and $O(1)$ NEXT and COUNT operations. Both use an efficient MaxHeap.

Space Complexity: Both use $O(n)$ space for storing emails and $O(1)$ auxiliary space for heap operations. Neither uses recursive heap functions.

Maintainability: Google Search has more descriptive function names and detailed comments. Perplexity has shorter function names and fewer comments. Both use global variables, which could be improved by placing the heap in a structure.

3. GenAI Used

I selected the Perplexity program as the basis for my assignment. It correctly implements a list based MaxHeap and efficiently prioritizes emails by category and date. Its separate heap functions make the code easy to understand and modify. It also allows me to write the comments as opposed to the Google Search code that already has comments.

4. Improvements

Correctness: I added the `trim()` command to multiple areas that needed it to remove spaces that may affect the code.

Execution Time: Improved `compare()` function by calculating integer values for priority and date once when the EMAIL command is processed. Reduced time complexity from heap operation to a constant time complexity.

Space Complexity: Space complexity was heavily optimized by the GenAI. Further improvements to space complexity could affect the overall integrity of the rest of the code. It was a calculated decision to keep the Space Complexity as is.

Maintainability: Added prologue comments and added inline comments to each line to better the readability of the code. Inline comments are added to each line to help analyze each line and explain to the reader what the code is doing.

5. Raw Code

This is the code that immediately came out of my prompt. The same prompt was used for both GenAI which is just the standalone [[Assignment 2 Instructions.pdf](#)] upload followed by a prompt “lets get started” for both Programs.

Raw Google Search AI Code

```
/* *****
*****
* Name of Program: EECS 348 Assignment 2 - CEO Email Priority Queue
* Brief Description: Implements a list-based MaxHeap from scratch
to manage and
*
*           prioritize emails for a company CEO.
Prioritizes by sender
*
*           category (Boss > Subordinate > Peer >
ImportantPerson >
*
*           OtherPerson) and resolves ties by picking the
newest date.
* Inputs: Standard input or a redirected test file containing
commands:
*
*           EMAIL, NEXT, READ, and COUNT.
* Output: Terminal printouts formatted exactly to the assignment
specification.
* All Collaborators: None
* Other Sources: ChatGPT / CoPilot used as a baseline for
structural comparison.
* Author's Full Name: [Your Name]
* Creation Date: September 17, 2026
* Revision Date: September 17, 2026
* Revisions: Initial complete implementation with strict edge case
handling.

*****
******/

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_EMAILS 1000
#define MAX_STR_LEN 256

/* Structure representing a single email entry */
typedef struct {
    char sender[MAX_STR_LEN];    /* Stores the raw string category
of the sender */
```

```

    char subject[MAX_STR_LEN];    /* Stores the email subject line
*/
    char date[MAX_STR_LEN];       /* Stores the raw date string
(MM-DD-YYYY) */
    int category_priority;        /* Integer rank: Boss=5 down to
OtherPerson=1 */
    long long date_numeric;       /* Compressed numeric
representation YYYYMMDD for fast comparison */
} Email;

/* Global array-based MaxHeap structure */
Email heap[MAX_EMAILS];
int heap_size = 0;

/**
 * Converts a string category into an integer priority value.
 * Higher values indicate higher priority.
 */
int get_category_priority(const char *category) {
    if (strcmp(category, "Boss") == 0) return 5;
    if (strcmp(category, "Subordinate") == 0) return 4;
    if (strcmp(category, "Peer") == 0) return 3;
    if (strcmp(category, "ImportantPerson") == 0) return 2;
    if (strcmp(category, "OtherPerson") == 0) return 1;
    return 0; /* Default case for unrecognized categories */
}

/**
 * Converts an MM-DD-YYYY date string into a comparable numeric
value (YYYYMMDD).
 */
long long parse_date_to_numeric(const char *date_str) {
    int month = 0, day = 0, year = 0;
    /* Extract integers using sscanf format matching */
    sscanf(date_str, "%d-%d-%d", &month, &day, &year);
    /* Combine into YYYYMMDD layout for direct
greater-than/less-than logic */
    return ((long long)year * 10000) + (month * 100) + day;
}

```

```

/**
 * Compares two emails to determine which has higher priority.
 * Returns > 0 if a has higher priority than b, < 0 if lower, 0 if
identical.
 */
int compare_emails(Email a, Email b) {
    /* Rule 1: Check category priority first */
    if (a.category_priority != b.category_priority) {
        return a.category_priority - b.category_priority;
    }
    /* Rule 2: If categories match, prioritize the newer date */
    if (a.date_numeric != b.date_numeric) {
        return (a.date_numeric > b.date_numeric) ? 1 : -1;
    }
    return 0;
}

/**
 * Swaps two Email structures in the heap array.
 */
void swap(int i, int j) {
    Email temp = heap[i];
    heap[i] = heap[j];
    heap[j] = temp;
}

/**
 * Restores the MaxHeap property by moving a node up the tree.
 */
void heapify_up(int index) {
    /* Continue swapping with parent until root is reached or heap
property is satisfied */
    while (index > 0) {
        int parent = (index - 1) / 2;
        if (compare_emails(heap[index], heap[parent]) > 0) {
            swap(index, parent);
            index = parent;
        } else {
            break;
        }
    }
}

```

```

    }
}

/**
 * Restores the MaxHeap property by moving a node down the tree.
 */
void heapify_down(int index) {
    while (2 * index + 1 < heap_size) {
        int left_child = 2 * index + 1;
        int right_child = 2 * index + 2;
        int largest = left_child;

        /* Check if right child exists and has a higher priority
        than the left child */
        if (right_child < heap_size &&
        compare_emails(heap[right_child], heap[left_child]) > 0) {
            largest = right_child;
        }

        /* If the largest child is greater than the current node,
        swap and descend */
        if (compare_emails(heap[largest], heap[index]) > 0) {
            swap(index, largest);
            index = largest;
        } else {
            break;
        }
    }
}

/**
 * Inserts a new email into the priority queue MaxHeap.
 */
void insert_email(Email email) {
    if (heap_size >= MAX_EMAILS) return; /* Guard against overflow
    */
    heap[heap_size] = email;             /* Place new item at the
    end of the list */
    heapify_up(heap_size);                /* Bubble up to restore
    heap invariant */
}

```

```

        heap_size++;                                /* Increment tracking
counter */
    }

    /**
     * Deletes the highest priority email from the MaxHeap without
printing it.
     */
void pop_email() {
    if (heap_size <= 0) return;                    /* Guard against popping
from an empty queue */
    heap[0] = heap[heap_size - 1];                /* Move the final element
to the root position */
    heap_size--;                                    /* Shrink the size metrics
*/
    heapify_down(0);                                /* Bubble down root element
to fix invariant */
}

int main() {
    char line[1024];

    /* Continuously parse input line-by-line until EOF */
    while (fgets(line, sizeof(line), stdin)) {
        /* Strip trailing newlines or carriage returns for safe
token comparisons */
        line[strcspn(line, "\r\n")] = 0;

        if (strncmp(line, "EMAIL ", 6) == 0) {
            Email new_email;
            /* Extract data content skipping the 6 character prefix
'EMAIL ' */
            char *data_part = line + 6;

            /* Parse comma-delimited fields sequentially */
            char *category_tok = strtok(data_part, ",");
            char *subject_tok = strtok(NULL, ",");
            char *date_tok = strtok(NULL, ",");

            if (category_tok && subject_tok && date_tok) {

```

```

        strcpy(new_email.sender, category_tok);
        strcpy(new_email.subject, subject_tok);
        strcpy(new_email.date, date_tok);

        /* Compute operational sorting fields */
        new_email.category_priority =
get_category_priority(category_tok);
        new_email.date_numeric =
parse_date_to_numeric(date_tok);

        /* Insert newly formed object into the custom
priority structure */
        insert_email(new_email);
    }
}
else if (strcmp(line, "COUNT") == 0) {
    printf("There are %d emails to read.\n", heap_size);
}
else if (strcmp(line, "NEXT") == 0) {
    if (heap_size > 0) {
        printf("Next email:\n");
        printf("Sender: %s\n", heap[0].sender);
        printf("Subject: %s\n", heap[0].subject);
        printf("Date: %s\n", heap[0].date);
    }
    /* Silently handle if heap is empty per requirements
edge cases */
}
else if (strcmp(line, "READ") == 0) {
    pop_email();
}
}
return 0;
}

```

Raw Perplexity Code

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

```

```
#define MAX_EMAILS 1000
#define MAX_STR 256

typedef struct {
    char sender[64];
    char subject[256];
    char date[16];
    int priority;
    int dateKey;
} Email;

Email heap[MAX_EMAILS];
int heapSize = 0;

int getCategoryPriority(char* category) {
    if (strcmp(category, "Boss") == 0) return 5;
    if (strcmp(category, "Subordinate") == 0) return 4;
    if (strcmp(category, "Peer") == 0) return 3;
    if (strcmp(category, "ImportantPerson") == 0) return 2;
    if (strcmp(category, "OtherPerson") == 0) return 1;
    return 0;
}

int parseDateKey(char* date) {
    int month, day, year;
    sscanf(date, "%d-%d-%d", &month, &day, &year);
    return year * 10000 + month * 100 + day;
}

int compare(Email a, Email b) {
    if (a.priority != b.priority) return a.priority - b.priority;
    return a.dateKey - b.dateKey;
}

void swap(Email* a, Email* b) {
    Email temp = *a;
    *a = *b;
    *b = temp;
}
```

```

void heapifyUp(int index) {
    while (index > 0) {
        int parent = (index - 1) / 2;
        if (compare(heap[index], heap[parent]) > 0) {
            swap(&heap[index], &heap[parent]);
            index = parent;
        } else {
            break;
        }
    }
}

void heapifyDown(int index) {
    while (1) {
        int left = 2 * index + 1;
        int right = 2 * index + 2;
        int largest = index;
        if (left < heapSize && compare(heap[left], heap[largest]) >
0)
            largest = left;
        if (right < heapSize && compare(heap[right], heap[largest])
> 0)
            largest = right;
        if (largest != index) {
            swap(&heap[index], &heap[largest]);
            index = largest;
        } else {
            break;
        }
    }
}

void insertEmail(Email e) {
    if (heapSize >= MAX_EMAILS) return;
    heap[heapSize] = e;
    heapifyUp(heapSize);
    heapSize++;
}

Email* peekEmail() {

```

```

        if (heapSize == 0) return NULL;
        return &heap[0];
    }

Email removeEmail() {
    Email empty = {"", "", "", 0, 0};
    if (heapSize == 0) return empty;
    Email top = heap[0];
    heap[0] = heap[heapSize - 1];
    heapSize--;
    if (heapSize > 0) heapifyDown(0);
    return top;
}

void trim(char* str) {
    int len = strlen(str);
    while (len > 0 && (str[len-1] == '\n' || str[len-1] == '\r' ||
str[len-1] == ' ')) {
        str[--len] = '\0';
    }
    int start = 0;
    while (str[start] == ' ') start++;
    if (start > 0) {
        memmove(str, str + start, len - start + 1);
    }
}

int main(int argc, char* argv[]) {
    FILE* file = fopen(argv[1], "r");
    if (!file) {
        printf("Could not open file.\n");
        return 1;
    }

    char line[512];
    while (fgets(line, sizeof(line), file)) {
        trim(line);
        if (strlen(line) == 0) continue;

        if (strncmp(line, "EMAIL", 5) == 0) {

```

```

    char* rest = line + 6;
    char* comma1 = strchr(rest, ',');
    if (!comma1) continue;
    *comma1 = '\0';
    char* category = rest;
    char* rest2 = comma1 + 1;
    char* comma2 = strrchr(rest2, ',');
    if (!comma2) continue;
    *comma2 = '\0';
    char* subject = rest2;
    char* date = comma2 + 1;

    Email e;
    strcpy(e.sender, category);
    strcpy(e.subject, subject);
    strcpy(e.date, date);
    e.priority = getCategoryPriority(category);
    e.dateKey = parseDateKey(date);
    insertEmail(e);
} else if (strcmp(line, "NEXT") == 0) {
    Email* e = peekEmail();
    if (e) {
        printf("Next email:\n");
        printf("Sender: %s\n", e->sender);
        printf("Subject: %s\n", e->subject);
        printf("Date: %s\n", e->date);
    } else {
        printf("No emails to read.\n");
    }
} else if (strcmp(line, "READ") == 0) {
    removeEmail();
} else if (strcmp(line, "COUNT") == 0) {
    printf("There are %d emails to read.\n", heapSize);
}

}

fclose(file);
return 0;
}

```